# AI & AC

# ASSIGNMENT-23.1

**HT :2303A51690**

**BATCH:28**

## Task 1 – Project Proposal

## PROMPT:

Prepare a project proposal for an AI-assisted application e.g e-commerce billing system. Include objectives, features, tools/technologies to be used, and expected outcome.

## OUTPUT:

lab23-1.py > ...

```
#Prepare a project proposal for an AI-assisted application (e.g., e-commerce billing system). Include objectives, features, tools/technologies to be used, and expected
Project Proposal: AI-Assisted E-commerce Billing System
Objectives:
1. Automate billing processes for e-commerce platforms.
2. Enhance customer experience by providing personalized recommendations and insights.
3. Reduce errors and improve efficiency in billing operations.
Features:
1. Automated Invoice Generation: The system will automatically generate invoices based on customer orders and payment information.
2. Personalized Recommendations: The AI will analyze customer purchase history and provide personalized product recommendations during the
billing process.
3. Fraud Detection: The system will use AI algorithms to detect and prevent fraudulent transactions.
Tools/Technologies:
1. Python: For developing the backend of the application.
2. TensorFlow/PyTorch: For implementing AI algorithms for recommendations and fraud detection.
3. Django/Flask: For building the web application interface.
4. SQL/NoSQL Database: For storing customer data, order information, and billing records
5. RESTful API: For integrating the billing system with e-commerce platforms.
Expected Outcome:
1. A fully functional AI-assisted billing system that can be integrated with various e-commerce platforms.
2. Improved customer satisfaction through personalized recommendations and faster billing processes.
3. Increased efficiency and reduced errors in billing operations, leading to cost savings for e-commerce businesses.
4. Enhanced security and fraud prevention measures, protecting both customers and businesses from fraudulent activities.

```

## EXPLANATION:

- This task is about creating a project idea to solve a real problem.
- It explains the objective, features, tools, and expected results.
- AI features like recommendations and fraud detection are included.
- It gives a basic plan for building the application.

## Task 2 – Application Design

## PROMPT:

design the system architecture diagrams and define the dabase schemas if needed and create a wirefrrames/ui mockups for frontend design

## OUTPUT:

```
#design the system architecture diagrams and define the dabase schemas if neede and create a wirefrrames/ui mockups for frontend design
System Architecture :
1. User Interface: A web-based interface where customers can view their orders, receive recommendations, and manage their billing informatior
2. Backend Server: A server that handles the business logic, processes billing information, and interacts
with the database and AI models.
3. AI Models: Separate modules for recommendation and fraud detection that process data and provide insights to the backend server.
4. Database: A structured database to store customer information, order details, billing records, and AI model outputs.
Database Schema:
1. Customers Table:
   - customer_id (Primary Key)
   - name
   - email
   - phone_number
   - address
2. Orders Table:
   - order_id (Primary Key)
   - customer_id (Foreign Key)
   - product_id
   - quantity
   - price
   - order_date
3. Billing Table:
   - billing_id (Primary Key)
   - order_id (Foreign Key)
   - total_amount
   - billing_date
4. Recommendations Table:
   - recommendation_id (Primary Key)
   - customer_id (Foreign Key)
   - product_id
   - recommendation_date
5. Fraud Detection Table:
```

```
   - quantity
   - price
   - order_date
3. Billing Table:
   - billing_id (Primary Key)
   - order_id (Foreign Key)
   - total_amount
   - billing_date
4. Recommendations Table:
   - recommendation_id (Primary Key)
   - customer_id (Foreign Key)
   - product_id
   - recommendation_date
5. Fraud Detection Table:
   - fraud_id (Primary Key)
   - order_id (Foreign Key)
   - fraud_score
   - detection_date
UI Mockups:
1. Home Page: Displays a welcome message, customer information, and a summary of recent orders
2. Billing Page: Shows detailed billing information for each order, including total amount, billing date, and payment status.
3. Recommendations Page: Displays personalized product recommendations based on the customer's purchase history.
4. Fraud Detection Page: Provides insights into potential fraudulent activities, including fraud scores and detection dates.
```

## EXPLANATION:

- This task focuses on designing how the application will work.
- It includes system architecture, database schema, and UI design.
- It shows how frontend, backend, and database are connected.
- It helps in planning the structure before coding.

**Task 3 – AI-Assisted Code Development**

**PROMPT:**

Implement the application in phase:

- Frontend: Build a simple UI using HTML, CSS, and JavaScript (login page, dashboard, and main functionality).

- Backend: Create RESTful APIs with proper server logic and user authentication.

- Database: Design schema and provide SQL queries for storing data.

- Integration: Connect frontend and backend using API calls, include authentication, error handling, and proper data flow.

Ensure the code is simple, well-structured, and includes comments for understanding.

Frontend (HTML/CSS/JavaScript)

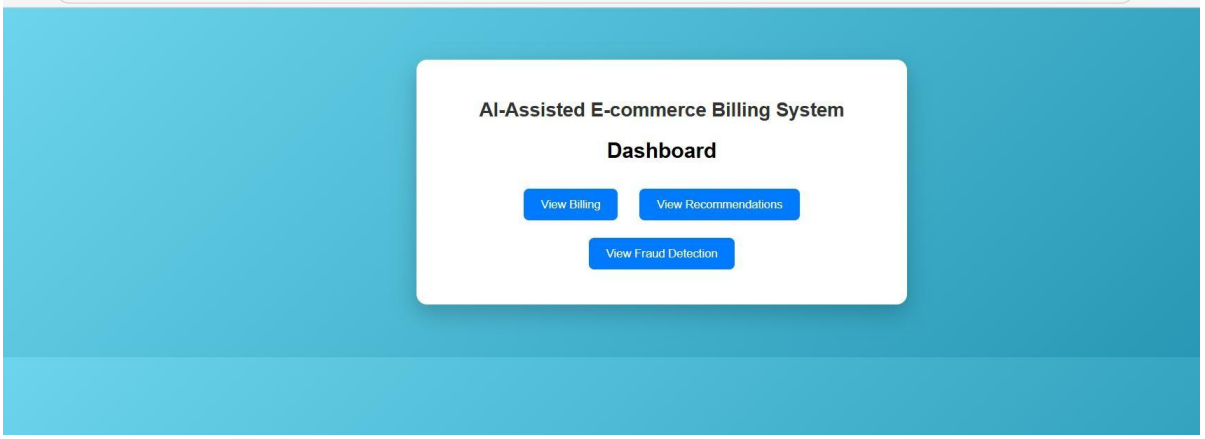
**CODE:**

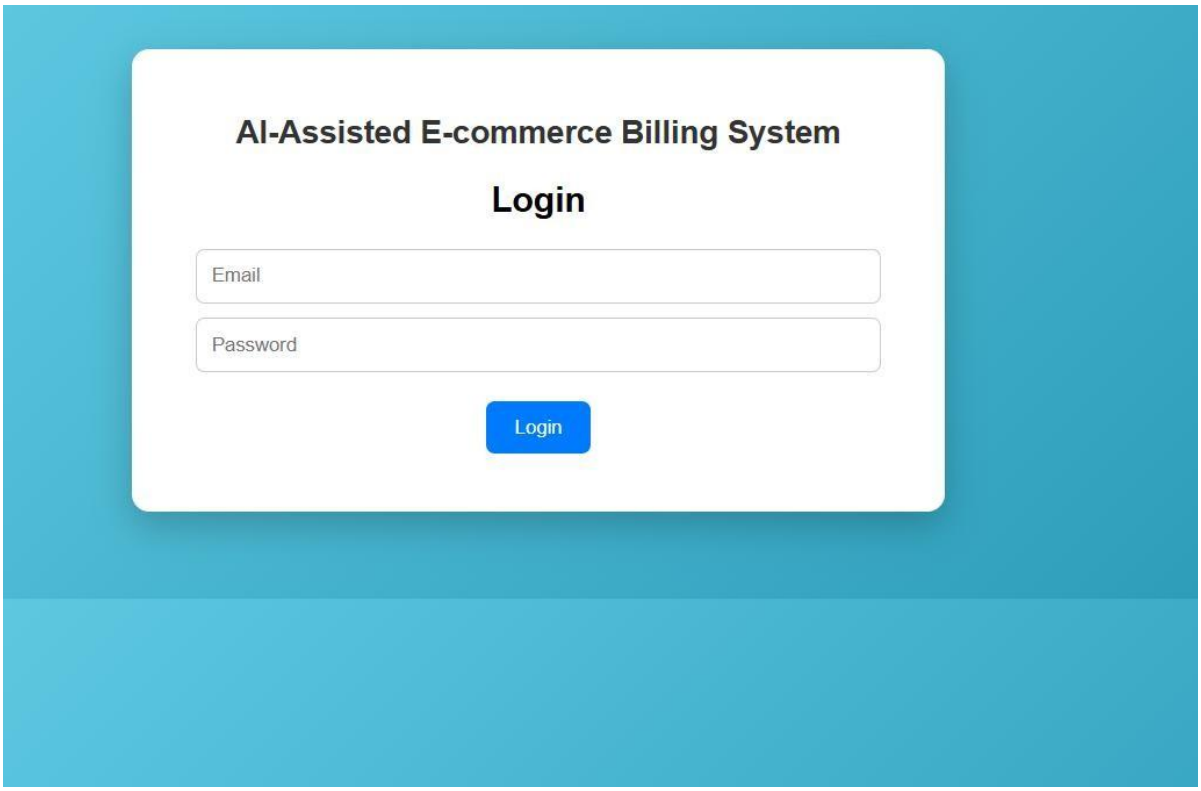
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI-Assisted E-commerce Billing System</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #6dd5ed, #2193b0);
            margin: 0;
            padding: 0;
        }

        .container {
            width: 80%;
            max-width: 500px;
            margin: 60px auto;
            padding: 30px;
            background-color: #fff;
            box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
            text-align: center;
            border-radius: 12px;
        }

        h1 {
            font-size: 22px;
            color: #333;
        }

        .login-form {
            margin-top: 20px;
        }

        .login-form input {
            display: block;
```

```html
<body>
    <div class="container">
        <h1>AI-Assisted E-commerce Billing System</h1>

        <!-- Login -->
        <div class="login-form">
            <h2>Login</h2>
            <input type="text" id="email" placeholder="Email">
            <input type="password" id="password" placeholder="Password">
            <button onclick="login()">Login</button>
        </div>

        <!-- Dashboard -->
        <div class="dashboard">
            <h2>Dashboard</h2>
            <button onclick="viewBilling()">View Billing</button>
            <button onclick="viewRecommendations()">View Recommendations</button>
            <button onclick="viewFraudDetection()">View Fraud Detection</button>
        </div>
    </div>

    <script>
        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function login() {
            const email = document.getElementById('email').value;
            const password = document.getElementById('password').value;

            if (email === 'admin@gmail.com' && password === '1234') {
                document.querySelector('.login-form').style.display = 'none';
                document.querySelector('.dashboard').style.display = 'block';
            } else {
                alert('Invalid email or password');
            }
        }
```

```
    <script>
        function login() {

            if (email === 'admin@gmail.com' && password === '1234') {
                document.querySelector('.login-form').style.display = 'none';
                document.querySelector('.dashboard').style.display = 'block';
            } else {
                alert('Invalid email or password');
            }
        }

        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function viewBilling() {
            // Dummy data (since no backend)
            const data = {
                total: 2500,
                items: ["Laptop", "Mouse"]
            };
            alert('Billing Information:\n' + JSON.stringify(data, null, 2));
        }

        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function viewRecommendations() {
            const data = ["Keyboard", "Headphones", "USB Hub"];
            alert('Recommendations:\n' + JSON.stringify(data, null, 2));
        }

        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function viewFraudDetection() {
            const data = { status: "No suspicious activity detected" };
            alert('Fraud Detection:\n' + JSON.stringify(data, null, 2));
        }
    </script>
</body>
</html>
```

## OUTPUT:






```
C:\Users\Pavani\Downloads\Assignmemt-\backend
import sqlite3

app = Flask(__name__)

Windsurf: Refactor | Explain | Generate Docstring | ×
def init_db():
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()

    cursor.execute('''
    CREATE TABLE IF NOT EXISTS Billing (
        billing_id INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id INTEGER,
        total_amount REAL,
        billing_date TEXT
    )
    ''')

    cursor.execute("DELETE FROM Billing")

    cursor.execute('''
    INSERT INTO Billing (order_id, total_amount, billing_date)
    VALUES (1, 1000, '2024-06-01')
    ''')

    cursor.execute('''
    INSERT INTO Billing (order_id, total_amount, billing_date)
```

```python
def init_db():
    INSERT INTO Billing (order_id, total_amount, billing_date)
    VALUES (2, 2000, '2024-06-02')
    ''')

    conn.commit()
    conn.close()

Windsurf: Refactor | Explain | Generate Docstring | ×
@app.route('/api/billing')
def get_billing():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    data = conn.execute('SELECT * FROM Billing').fetchall()
    conn.close()
    return jsonify([dict(row) for row in data])

Windsurf: Refactor | Explain | Generate Docstring | ×
@app.route('/')
def home():
    return "Server is running!"

if __name__ == '__main__':
    init_db()
    app.run(debug=False)
```

Pretty-print ☑


```
[
  {
    "billing_date": "2024-06-01",
    "billing_id": 1,
    "order_id": 1,
    "total_amount": 1000
  },
  {
    "billing_date": "2024-06-02",
    "billing_id": 2,
    "order_id": 2,
    "total_amount": 2000
  }
]
```

**EXPLANATION:**

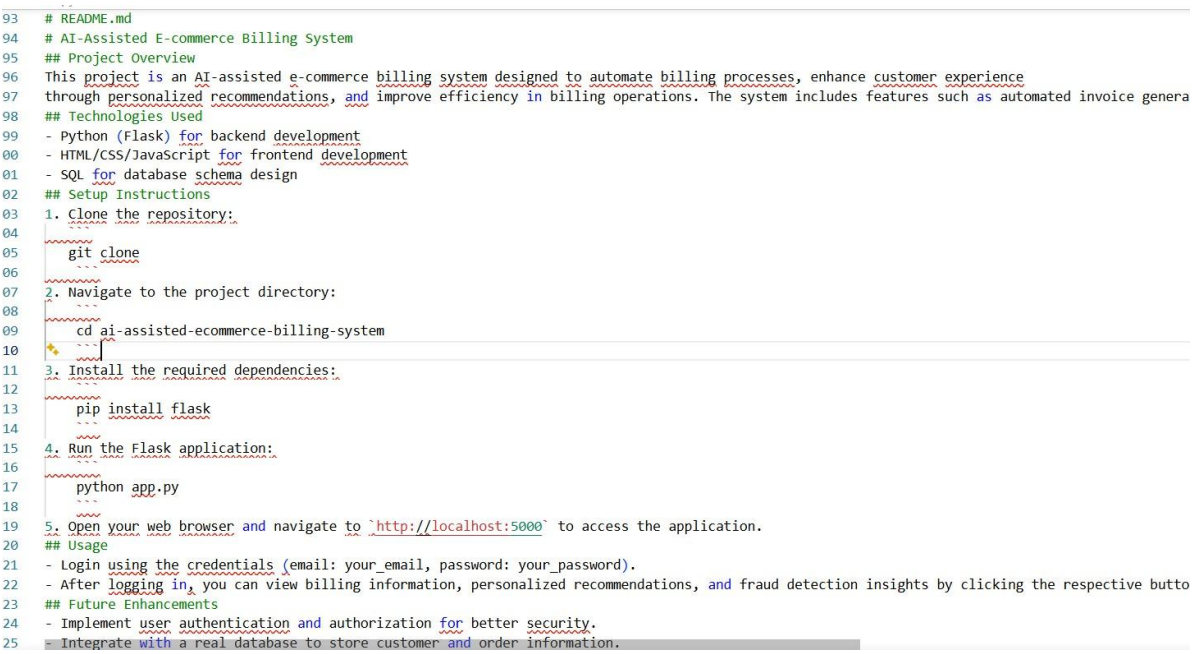
- This task involves creating the user interface using HTML, CSS, and JavaScript.
- It includes pages like login, dashboard, and main features.
- The design should be simple, user-friendly, and easy to use.
- It connects with backend APIs to display data and perform actions.

## Task 4 – Documentation & Code Quality

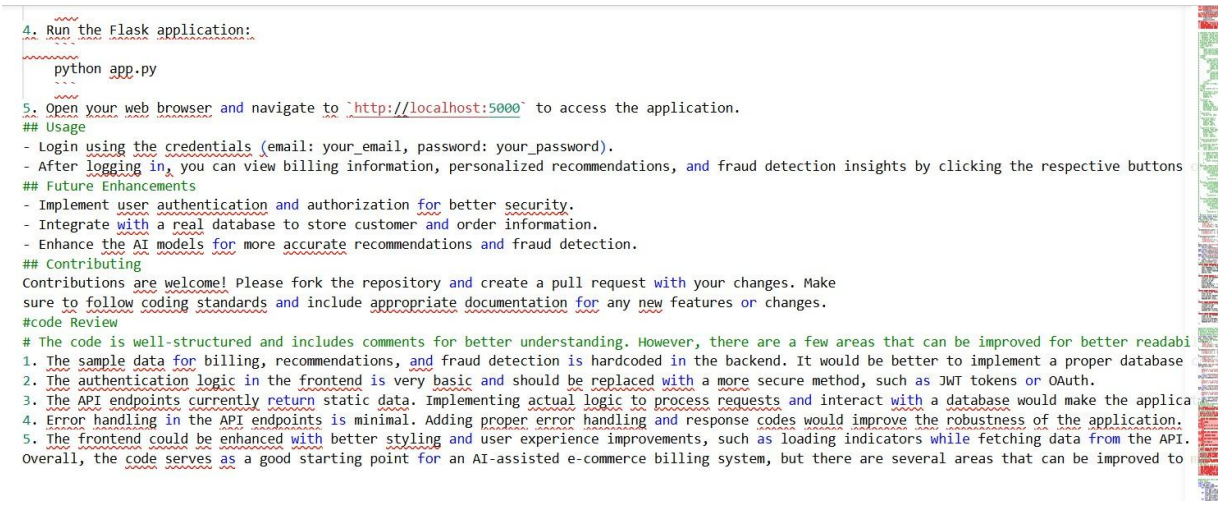
**PROMPT:**

generate automatic documentation and line comments using ai and ensure clear Readme.md for project setup and usage conduct Ai-assisted code review to refine readability and maintainability of the codebase, ensuring adherence to best practices and coding standards.

**OUTPUT:**



```
# README.md
# AI-Assisted E-commerce Billing System
## Project Overview
This project is an AI-assisted e-commerce billing system designed to automate billing processes, enhance customer experience
through personalized recommendations, and improve efficiency in billing operations. The system includes features such as automated invoice genera
## Technologies Used
- Python (Flask) for backend development
- HTML/CSS/JavaScript for frontend development
- SQL for database schema design
## Setup Instructions
1. Clone the repository:
   ```
   git clone
   ```
2. Navigate to the project directory:
   ```
   cd ai-assisted-ecommerce-billing-system
   ```
3. Install the required dependencies:
   ```
   pip install flask
   ```
4. Run the Flask application:
   ```
   python app.py
   ```
5. Open your web browser and navigate to `http://localhost:5000` to access the application.
## Usage
- Login using the credentials (email: your_email, password: your_password).
- After logging in, you can view billing information, personalized recommendations, and fraud detection insights by clicking the respective butto
## Future Enhancements
- Implement user authentication and authorization for better security.
- Integrate with a real database to store customer and order information.
```



```
4. Run the Flask application:

    python app.py

5. Open your web browser and navigate to `http://localhost:5000` to access the application.
## Usage
- Login using the credentials (email: your_email, password: your_password).
- After logging in, you can view billing information, personalized recommendations, and fraud detection insights by clicking the respective buttons
## Future Enhancements
- Implement user authentication and authorization for better security.
- Integrate with a real database to store customer and order information.
- Enhance the AI models for more accurate recommendations and fraud detection.
## Contributing
Contributions are welcome! Please fork the repository and create a pull request with your changes. Make
sure to follow coding standards and include appropriate documentation for any new features or changes.
#code Review
# The code is well-structured and includes comments for better understanding. However, there are a few areas that can be improved for better readabi
1. The sample data for billing, recommendations, and fraud detection is hardcoded in the backend. It would be better to implement a proper database
2. The authentication logic in the frontend is very basic and should be replaced with a more secure method, such as JWT tokens or OAuth.
3. The API endpoints currently return static data. Implementing actual logic to process requests and interact with a database would make the applica
4. Error handling in the API endpoints is minimal. Adding proper error handling and response codes would improve the robustness of the application.
5. The frontend could be enhanced with better styling and user experience improvements, such as loading indicators while fetching data from the API.
Overall, the code serves as a good starting point for an AI-assisted e-commerce billing system, but there are several areas that can be improved to
```

**EXPLANATION:**

- This task focuses on writing clear documentation for the project.
- AI is used to generate comments and improve code readability.
- A README file is created for setup and usage instructions.
- Code review is done to ensure good quality and best practices.

## Task 5 – Testing & Debugging

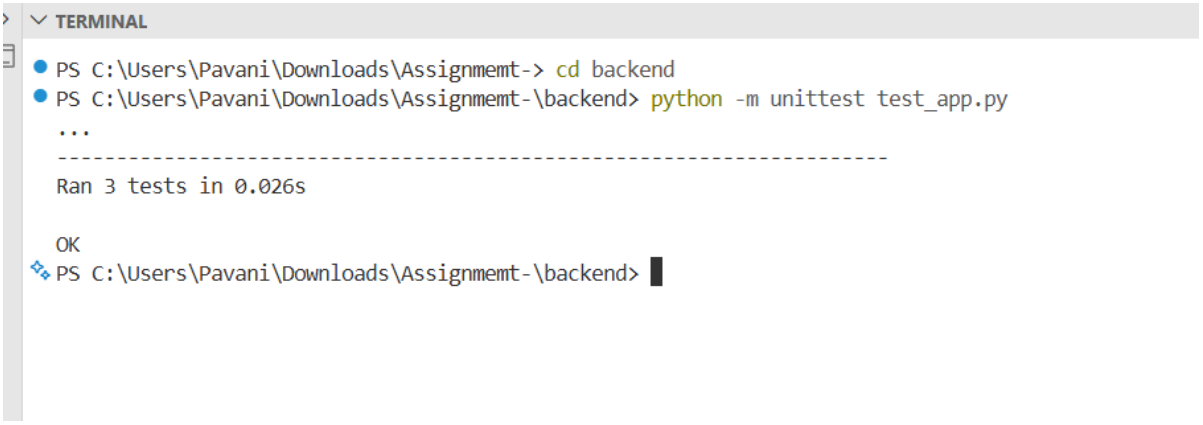
**PROMPT:**

Generate unit tests and identify vulnerabilities/ security flaws and suggest optimization for performance

**CODE:**

```python
# Unit tests
import unittest
from app import app
class TestBillingSystem(unittest.TestCase):
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def test_get_billing(self):
        response = self.app.get('/api/billing')
        self.assertEqual(response.status_code, 200)
        self.assertIn('total_amount', response.get_json())
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def test_get_recommendations(self):
        response = self.app.get('/api/recommendations')
        self.assertEqual(response.status_code, 200)
        self.assertIsInstance(response.get_json(), list)
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def test_get_fraud_detection(self):
        response = self.app.get('/api/fraud-detection')
        self.assertEqual(response.status_code, 200)
        self.assertIn('fraud_score', response.get_json())
if __name__ == '__main__':
    unittest.main()
```

**OUTPUT:**




**EXPLANATION:**

- This task involves creating unit tests to check if the application works correctly.
- AI is used to find bugs, security issues, and vulnerabilities in the code.
- It also suggests improvements to make the application faster and efficient.
- Overall, it ensures the application is reliable, secure, and well-optimized.

## Task 6 – Deployment & Presentation

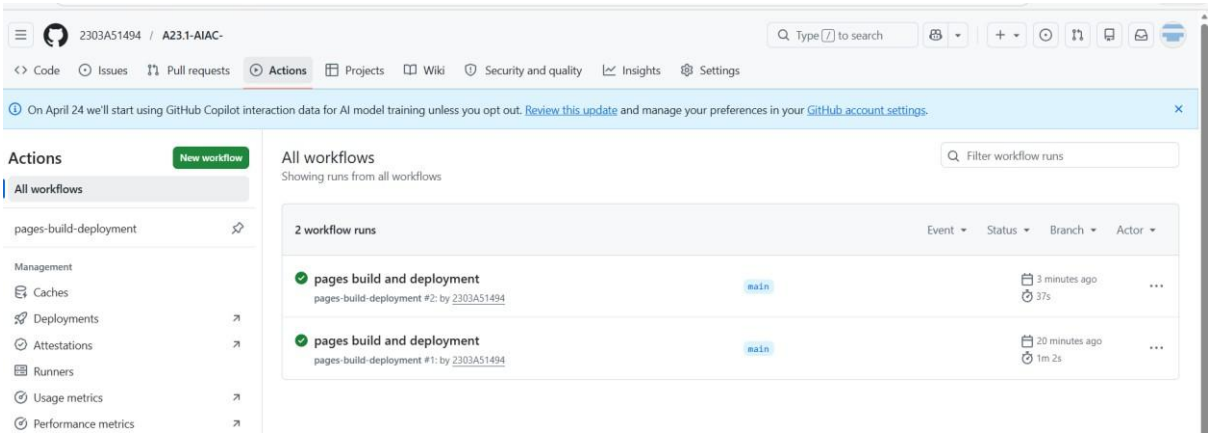
**PROMPT:**

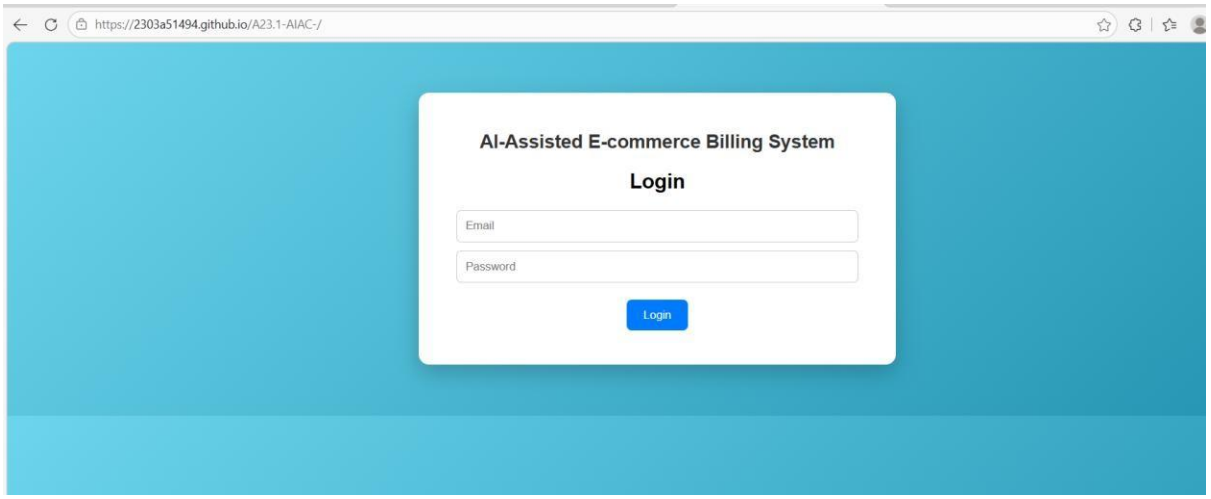
deploy application on a free hosting platform eg.Hroku github pages or firebase etc

**OUTPUT:**

```
PS C:\Users\Pavani\Downloads\Assignmemt-> git init
Initialized empty Git repository in C:/Users/Pavani/Downloads/Assignmemt-/.git/
PS C:\Users\Pavani\Downloads\Assignmemt-> git add .
PS C:\Users\Pavani\Downloads\Assignmemt-> git branch -M main
PS C:\Users\Pavani\Downloads\Assignmemt-> git remote add origin https://github.com/2303A51494/A23.1-AIAC-.git
```

```
PS C:\Users\Pavani\Downloads\Assignmemt-> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
```



**EXPLANATION:**

- This task involves deploying the application on a free platform like GitHub Pages.
- The frontend of the project is successfully hosted and accessible online.
- It allows users to view and interact with the application through a live link.
- Deployment makes the project easy to share and demonstrate.